



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

CE3006: Digital Communication Project Report

Group 11

Chew Loh Seng (U1921147L)

Royce Tan Chun Wei (U1921394L)

Seah Wei Hong (U1922097G)

Siti Nazhura Binte Muhamad Anuar (U1921339L)

Syahir Bin Toriman (U1921044H)

Contents

1. Introduction.....	3
2. Phase 1 – Data Generation	4
2.1. Introduction.....	4
2.2. Implementation & Experiment.....	4
2.3. Results	4
3. Phase 2 – Modulation for Communications.....	7
3.1. Introduction.....	7
3.2. Implementation & Experiment.....	7
3.2.1. Coherent OOK & Coherent BFSK equation to Matlab code	11
3.2.2. Coherent BPSK equation to Matlab version	11
3.3. Results	12
4. Phase 3 – Basic error control coding.....	19
4.1. Introduction.....	19
4.2. Implementation & Experiment.....	20
4.3. Results	21
5. Conclusion	26
6. References.....	27

1. Introduction

Digital communication systems are communication systems where information sent from one point to another is in the form of a digital signal. The nature of these systems are more immune to noise and distortion, which allows for efficient and reliable transmission even at long distances.

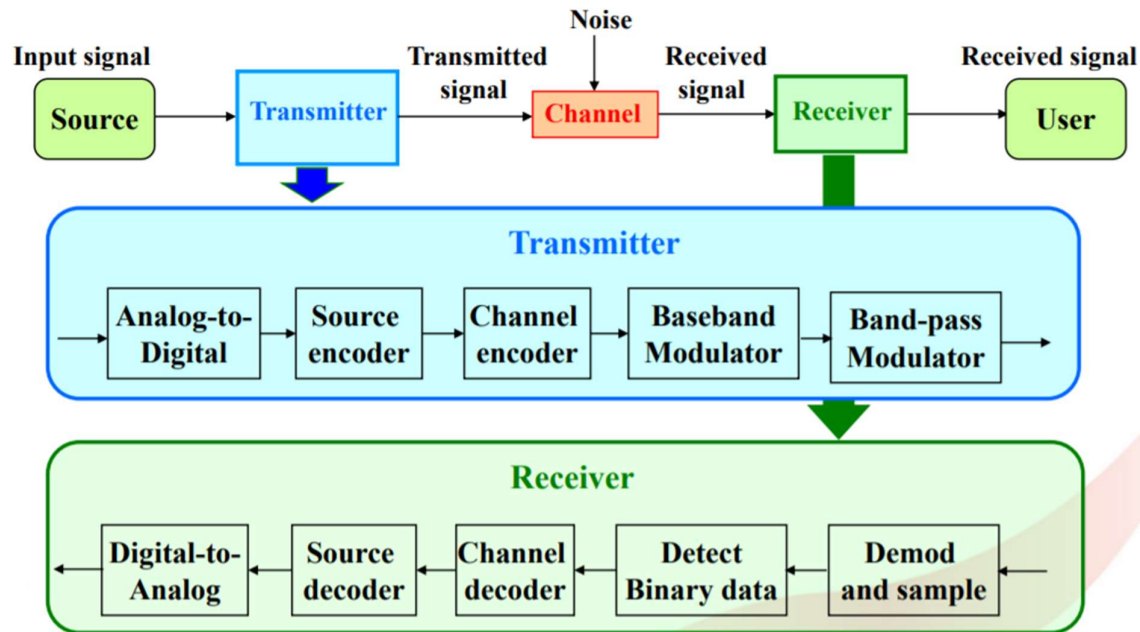


Figure 1 - Block diagram of a Digital Communication System

Figure 1 is a simple block diagram of a digital communication system. The block diagram shows the three main components of the system: Transmitter, Channel & Receiver. The transmitter consists of signal processing elements to ensure that the input signal is suitable, reliable and efficient for transmission. The channel is the medium that the signal gets transmitted through. Due to the physical property of the channel, it introduces noise and distorts the transmitted signal. The receiver consists of signal processing elements that converts and decodes, where necessary, the received signal to the original transmitted data.

Designing and simulating a digital communication system can be done with MATLAB. In this project, MATLAB is used to design the individual block diagrams while simulating different parameters of the digital communication system to better understand its effects. This project is divided into three phases: Phase 1 - Data Generation, Phase 2 - Modulation for Communication & Phase 3 - Basic Error Control Coding to Improve Performance. In each phase, the digital communication system is implemented and its performance is analysed. The metric used to assess our system performance will be the Bit Error Rate (BER). BER is computed by $\text{Bit Error Rate} = \frac{\text{Number of Bit Error}}{\text{Total Number of Transmission Bits}}$.

2. Phase 1 – Data Generation

2.1. Introduction

Phase 1 is to explore the effect of transmitting a Baseband data through an additive white Gaussian noise channel with varying Signal-To-Noise ratio (SNR). As the value of SNR increases, the effect of noise will decrease which leads to an increase in system performance. However in the physical world, an increase in SNR is mainly done by increasing the transmission power, and there are physical limitations to how much it can increase. In this project, physical limitations are not taken into consideration to better explore the various effects of SNR.

2.2. Implementation & Experiment

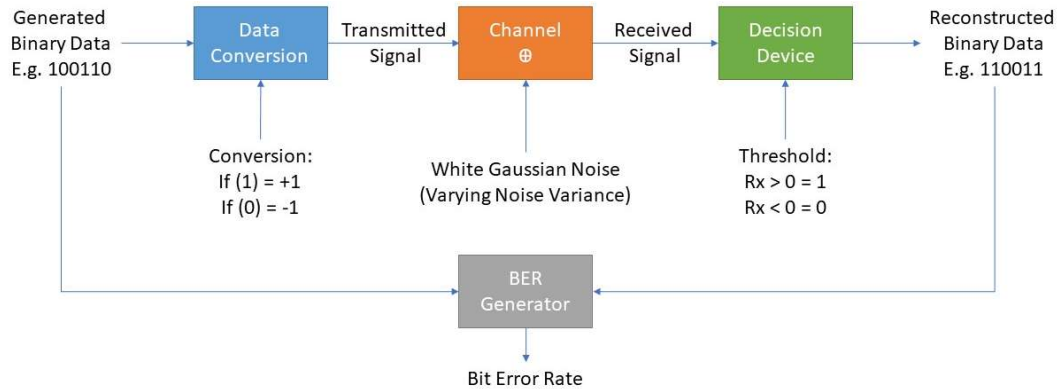


Figure 2.2.1 - Overview of Baseband data vs SNR values implementation

This phase of the project is implemented as according to figure 2.2.1. A random binary Baseband data signal of bit length 1024 bits is generated. The data signal then undergoes a data conversion whereby data 1 and 0 converts to +1 and -1 for transmission. White Gaussian noise of varying variance based on SNR value ranging from 0dB to 50dB at step size of 5dB is generated. The transmitted signal is added with the generated noise samples to produce a “received signal” at every SNR interval. The received signal then undergoes a decision device whereby data > 0 and data < 0 is reconstructed as 1 and 0 respectively. The reconstructed binary data is then compared with the initial generated binary data to compute the BER. This process is then run with a predefined number of test noise samples (ie., 100 samples) to compute the average BER for every SNR interval.

2.3. Results

This section will show the results of transmitting a baseband data through an additive white Gaussian noise channel with varying Signal-To-Noise ratio (SNR).

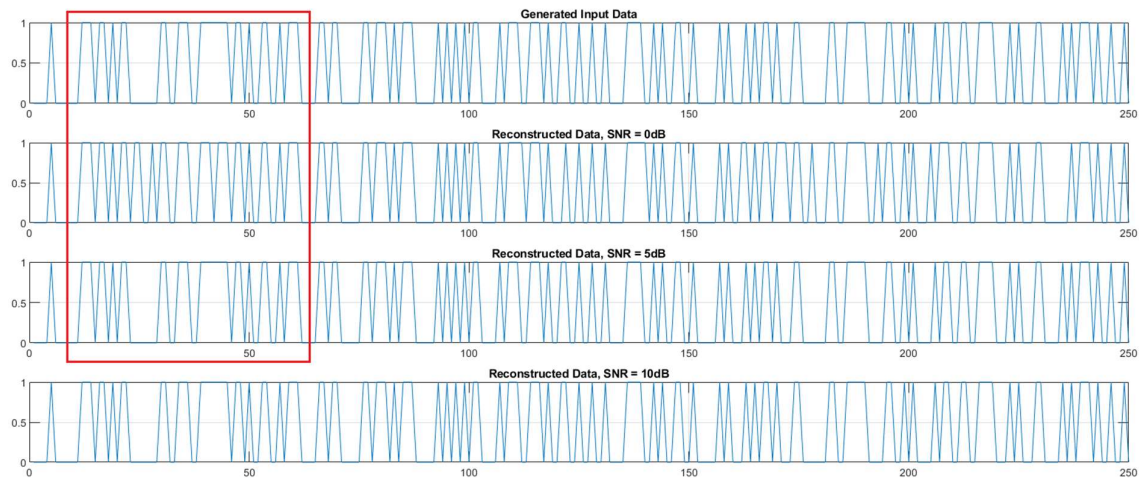


Figure 2.3.1 - Generated Input Data vs Reconstructed Data of varying SNR

Figure 2.3.1 above shows a comparison between the generated input data and the reconstructed data from varying SNR values. The data has been cut to show 200 bits out of the full bit length. It can be observed at SNR = 0dB, evident error is seen in the reconstructed data (highlighted in red) when compared with the generated input data. This error is not easily observed for SNR = 5dB & 10dB. Just by observing the snippet of 200 bits, it can be seen that reconstructed data of SNR > 5dB is able to generate the same data as the input. To better observe the performance of the system, the calculated BER is plotted alongside the theoretical BER.

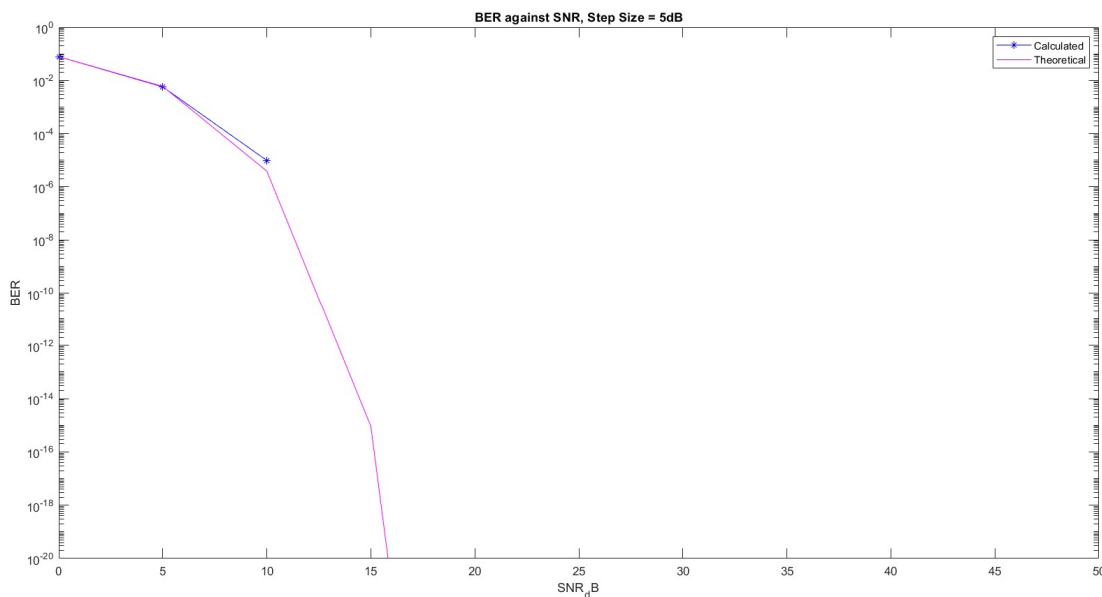


Figure 2.2.2 - BER against SNR for step size 5dB

Figure 2.2.2 shows the BER against SNR of the system for a range of **0 - 50dB with an interval of 5dB**. It can now be observed that bit error was generated for SNR = 5dB & 10dB and bit error is 0 for SNR > 10dB. The calculated BER shows similarity to the theoretical BER and that as SNR increases,

the BER decreases. However, at $\text{SNR} > 15\text{dB}$, it is difficult to observe the BER. As a result, in this project, the noise samples have been set in the **range of 0 - 15dB with an interval of 0.5dB** to be able to observe a smoother graph.

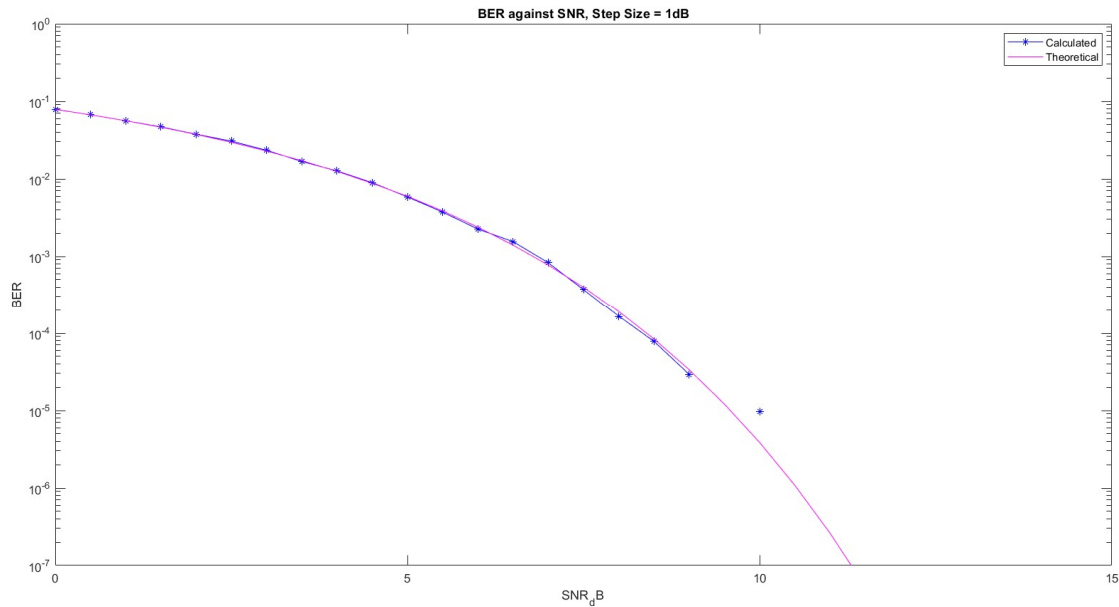


Figure 2.2.3 - BER against SNR for step size 1dB

Figure 2.2.3 shows the BER against SNR of the system for a range of **0 - 15dB with an interval of 0.5dB**. A decrease in the step size provides a smoother graph to better observe the effect of SNR against the BER. The calculated BER still shows similarity to the theoretical BER. For this project, the SNR of all the phases will be plotted from a range of **0 - 15dB with an interval of 0.5dB** to provide a smoother observation.

3. Phase 2 – Modulation for Communications

3.1. Introduction

Phase 2 is a bandpass process for our simple digital communication system. The process consists of:

- 1) Modulation
- 2) Demodulation and Detection

The main reason why we do bandpass modulation is that a baseband process sends over information without modulation. Bandpass modulation shifts the signal to a higher frequency before transmission, as it is robust against noise and interference, which is suitable to encode information from a message source and thus, this process helps us to modulate the amplitude, frequency, and phase. In the real world, bandpass modulation is used extensively as we are most likely to share channels with other sources, and different protocols require different frequencies.

For this project, we implemented three of the bandpass modulation techniques, namely, On-Off Keying (OOK), Binary Phase Shift Keying (BPSK), and Binary Frequency Shift Keying (BFSK). However, there are more modulation techniques but we will only focus on the aforementioned techniques.

3.2. Implementation & Experiment

For this experiment, Coherent ASK, BPSK, and BFSK will be implemented as shown in Figures 3.2.1, 3.2.2, and 3.2.3 respectively on MATLAB. The figures show the required steps that we have implemented for the coherent detection from start to end. The detailed steps will be explained in this section. The following paragraphs are the summaries of each step for coherent detection.

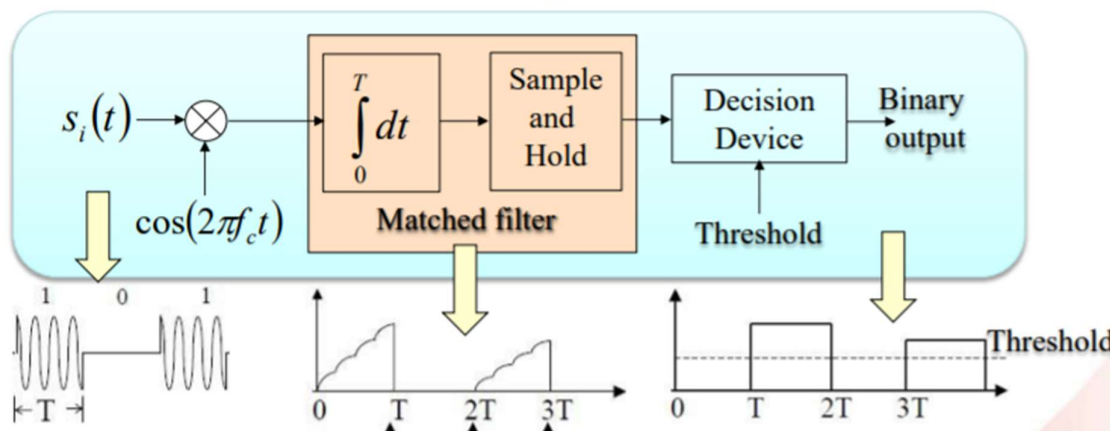


Figure 3.2.1 - Coherent ASK Implementation Diagram

Coherent ASK Implementation Steps:

1. Mixer - Multiplying carrier signal with the modulated signal
2. Matched filter - Low pass filter as an integral with sample and hold
3. Decision Device - Threshold detection

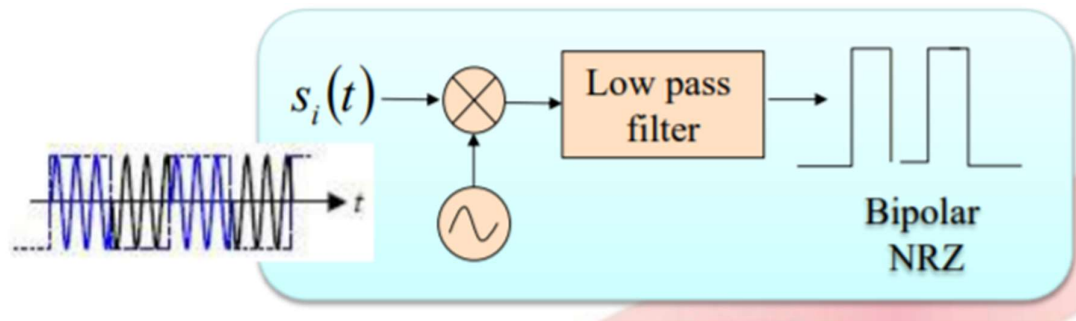


Figure 3.2.2 - Coherent BPSK Implementation Diagram

Coherent FSK Implementation Steps:

1. Mixer - Multiplying carrier signal and modulated signal
2. Low pass filter - Filter out high frequencies
3. Decision Device - Threshold detection

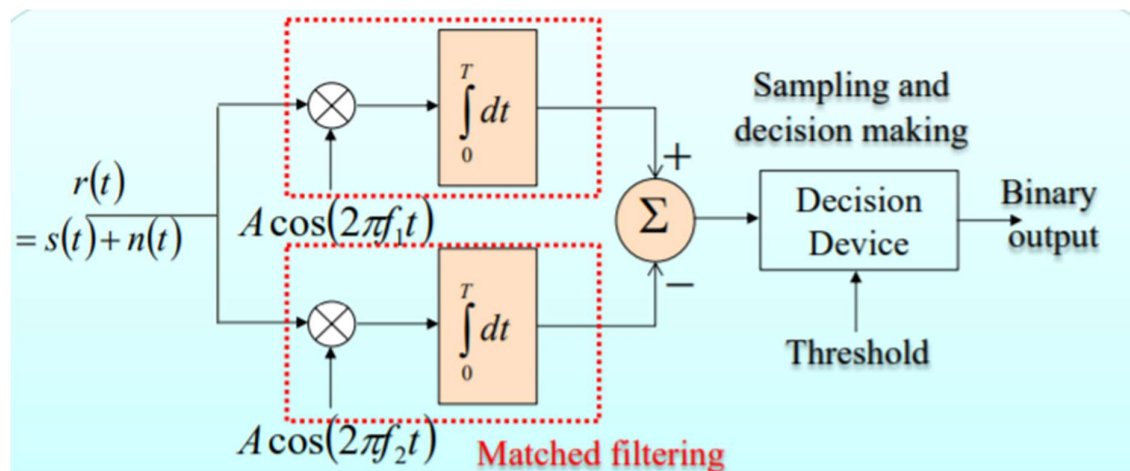


Figure 3.2.3 - Coherent BFSK Implementation Diagram

Coherent FSK Implementation Steps:

1. Mixer 1 - Multiplying carrier signal f_1 with the modulated signal
2. Mixer 2 - Multiplying carrier signal f_2 with the modulated signal
3. Matched filter 1 - Low pass filter (integral) with sample and hold for mixer 1 output
4. Matched filter 2 - Low pass filter (integral) with sample and hold for mixer 2 output
5. Summation - The difference between matched filter 1 and matched filter 2 output
6. Decision Device - Sample and threshold detection

For this experiment, we used a carrier frequency of 10kHz for OOK and BPSK but for the BFSK we used two carrier frequencies, 10kHz and 50kHz. Once we have initialized the carrier frequency, we can compute the carrier signal:

$$\text{Carrier signal} = \text{Amplitude} \times \cos(2\pi f_c t)$$

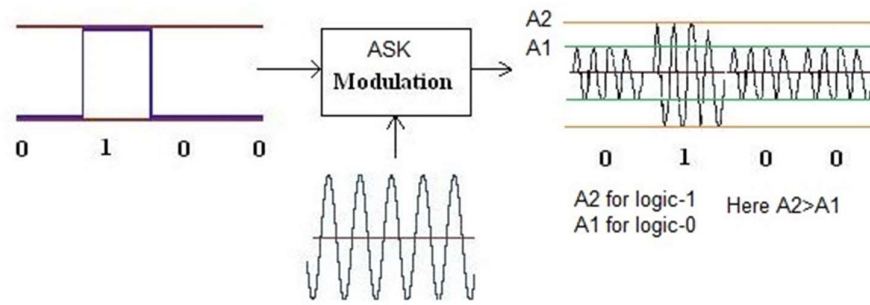
$$\text{Carrier signal (OOK, BPSK \& BFSK)} = 8 \times \cos(2\pi 10000t)$$

$$\text{Carrier signal (BFSK)} = 8 \times \cos(2\pi 50000t)$$

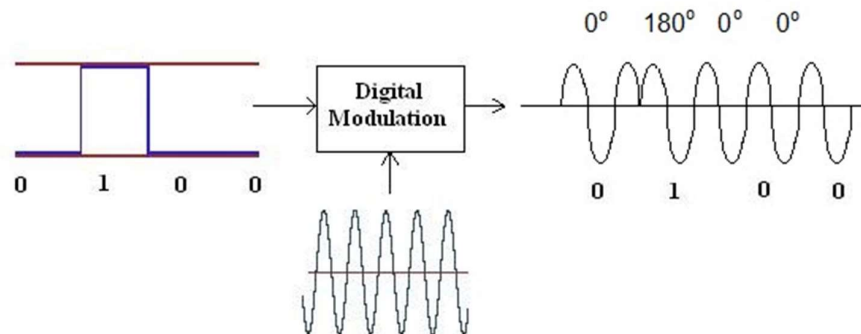
As mentioned, all three techniques use a carrier frequency of 10kHz, and it will determine the sampling frequency of a carrier signal which will be oversampled 16 times. The sampling frequency is thus used for modulation for the transmission, which has 1024 bits.

$$\text{Sampling frequency} = 16 \times 10\text{kHz} = 160\text{kHz}$$

Firstly, we implemented **On-Off Keying (OOK)**, which is the simpler form of Amplitude Shift Keying (ASK), which transmits signals that take in different amplitude values based on the data signal (in 1 or 0 formats). In addition, the phase and frequency will always remain constant but the amplitude always changes.

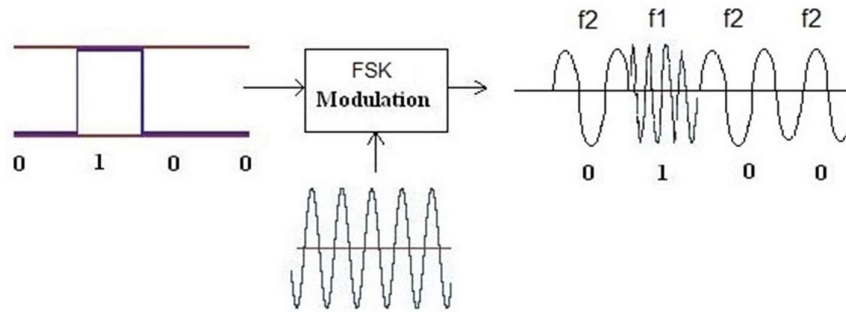


Binary Phase Shift Keying (BPSK) is the simpler form of Phase Shift Keying (PSK), which conveys data by changing the phase of a constant frequency reference signal based on the data signal (in 1 or -1 formats). PSK switches between 10kHz (0 degrees) and 10kHz (180 degrees) signals for all 'zero' to 'one' transitions occurring in the transmitted data stream. This also means that the frequency and amplitude will always remain constant but the phase always changes.



We also implemented **Binary Frequency Shift Keying (BFSK)**, which is the simpler form of Frequency Shift Keying (FSK). FSK transmits signals that take in two unique frequency values (in the project we used 10kHz and 50kHz) and the frequency changes according to the data signal. FSK

transitions from 10kHz to 50kHz for all ‘zero’ to ‘one’ transitions. Additionally, this technique is insensitive to amplitude and phase fluctuations in the channel.



For all the three modulated signals, we added additional white noise to simulate noise during our transmission with the required SNR values **ranging from 0dB to 15dB with an interval of 0.5dB**. At the receiver's end, all three modulated signals (with noise) will be demodulated and detected.

Demodulation is done after receiving the transmitted signal. The process is broken up into three parts, namely to multiply the received signal with the carrier signal, pass the received signal through a 6th order low-pass filter (a replacement for integrators) and lastly, sample and threshold to convert the message to binary data. Demodulation has two predominant methods, coherent detection and non-coherent detection. For this portion, we focused on doing the coherent detection method.

To do the multiplication of the carrier signal with the modulated (received) signal:

$$\text{Modulated Signal} = A_1 \cos(2\pi f_c t)$$

$$\text{Carrier Signal} = A_2 \cos(2\pi f_c t)$$

$$\text{Mixer Output} = \text{Modulated Signal} \times \text{Carrier Signal}$$

$$\text{Mixer output} = A_1 A_2 \cos^2(2\pi f_c t)$$

$$= A_1 A_2 \left[\frac{1 + \cos(4\pi f_c t)}{2} \right]$$

$$= \frac{A_1 A_2}{2} + \frac{A_1 A_2}{2} \cos(4\pi f_c t)$$

With the **mixer output**, the constant term and the cosine term are separated to allow the low-pass filter to retrieve the original data. However, for **BFSK**, there will be two mixer outputs as there are two carrier frequencies that are required for modulation, and we will need to get the difference between these two mixer outputs before feeding them into the low-pass filter.

$$\text{Mixer Output 1} = \text{Modulated Signal } f_1 \times \text{Carrier signal}$$

$$\text{Mixer Output 2} = \text{Modulated Signal } f_2 \times \text{Carrier signal}$$

$$\begin{aligned} \text{Difference (BFSK before feeding into low pass filter)} \\ = \text{Mixer Output 1} - \text{Mixer Output 2} \end{aligned}$$

Next, we passed the modulated signal through a 6th order low-pass filter with a cut-off frequency of 0.2. Passing the modulated signal through a low pass filter will filter out the higher frequencies and keep the lower frequencies, in this case, the higher frequencies are the cosine terms and the lower frequencies are the constant terms. Since the constant terms have 0 frequency, they can be used in phase 3, where they will be used to retrieve the original data. Furthermore, we can replace the low pass filters with integrators as they both filter out higher frequencies of a signal¹.

Lastly, we used the decision threshold logic for all three modulated signals. For this project, we implemented the measured and theoretical bit error rate calculations. The measured bit error rate is done by taking the bit error rates for all of the three modulated signals and taking the average with a predefined number of test noise samples (i.e., 100 samples), respectively. With 100 noise samples, we took the midpoint of the bits for the decision logic. This results in a smoother BER curve, thus it will be similar to the theoretical BER curve.

For the theoretical bit error rate calculation, all three techniques have respective equations for the coherent detection, as explained below:

3.2.1. Coherent OOK & Coherent BFSK equation to Matlab code

$$\begin{aligned}
 P(0) &= P(1) = \frac{1}{2} \\
 P(e|0) &= P(e|1) \\
 P(e) &= P(0)P(e|0) + P(1)P(e|1) \\
 P(e) &= \frac{1}{2}P(e|0) + \frac{1}{2}P(e|1) \\
 P(e) &= P(e|0) = Q\left(\sqrt{\frac{E_b}{N_o}}\right) \\
 Q\left(\sqrt{\frac{E_b}{N_o}}\right) &= \frac{1}{2} \operatorname{erfc}\left(\sqrt{\frac{E_b}{2N_o}}\right) - (1)
 \end{aligned}$$

3.2.2. Coherent BPSK equation to Matlab version

$$\begin{aligned}
 Q\left(\sqrt{\frac{2E_b}{N_o}}\right) &= \frac{1}{2} \operatorname{erfc}\left(\sqrt{\frac{E_b}{N_o}}\right) - (2) \\
 P(e) &= P(e|0) = Q\left(\sqrt{\frac{2E_b}{N_o}}\right)
 \end{aligned}$$

For coherent OOK and BPSK detections, E_b is represented as the average signal energy per bit and the N_o is represented as the noise power spectral density. Both techniques use the same equation (1) to determine the bit error rate. The N_o is represented by $\frac{\text{noise power of modulation technique}}{\text{data rate} * 2}$, where the noise power of the modulation technique is determined by $\frac{\text{signal power of modulation technique}}{SNR}$ and the data rate is 1 kbps, which is the same for all three techniques.

¹ <https://webhome.phy.duke.edu/~schol/phy271/faqs/faq8/node6.html>

As for E_b , it is represented as $E_b = E_1 + E_2$. The OOK data signals are only considered 1 or 0 bits and when the bit is 1, the E_1 will be represented as $\frac{A^2}{2}T$ and E_2 is represented as 0. After we obtained the values for E_1 and E_2 , E_b will be $\frac{A^2}{2}T + 0 = \frac{A^2}{2}T$, where T is the period. With N_O and E_b values obtained, the bit error rate can be computed from equation (1) as derived from above.

However, obtaining the BPSK and BFSK E_b is different from OOK. The data signal is always transmitted for these two modulation techniques regardless of the data format of the bits. Therefore, E_b for BPSK and BFSK will be $\frac{A^2}{2}T + \frac{A^2}{2}T = A^2T$. Similarly, with the N_O and E_b values obtained, the bit error rate can be computed from equation (2) as derived from above.

3.3. Results

This section will show all three modulated techniques of each state, which are generating random data for modulation, adding noise to modulating signals to simulate noise in the channel, and demodulation for detection to retrieve the original data. For the selected SNR value, we chose 5dB because it will be easier for us to study the signal transmission.

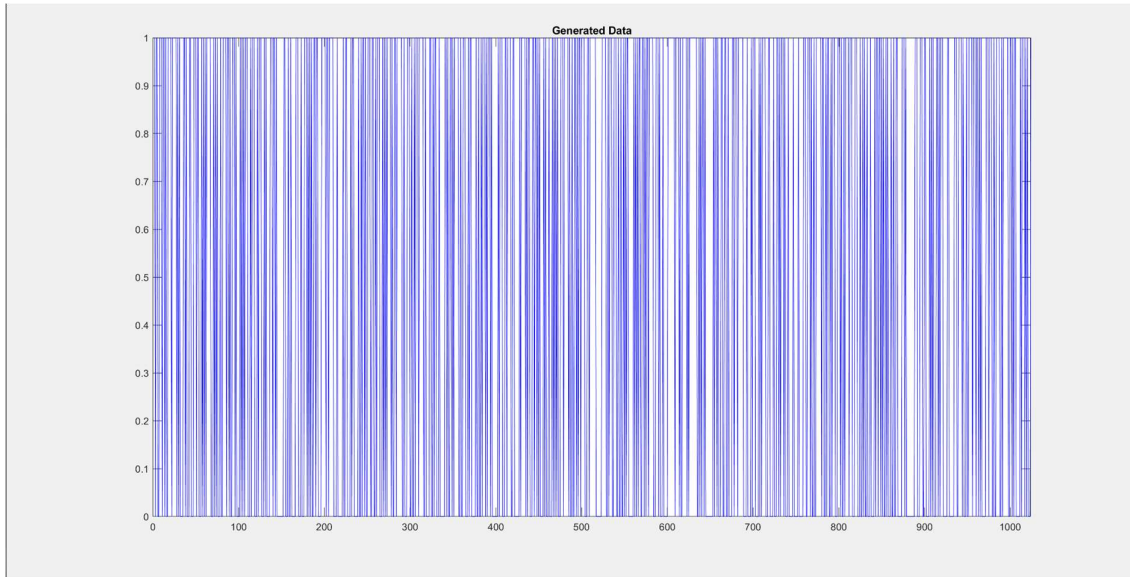


Figure 3.3.1 - Generated Data

The generated data is 1024 bits, as the bits for transmission is 1024 (from Phase 1). With the given carrier frequency of 10kHz and sampling frequency of 160kHz, the generated data will be oversampled at 160KHz, and it will be fed into the different selected modulation techniques, as seen in Figure 3.3.1.

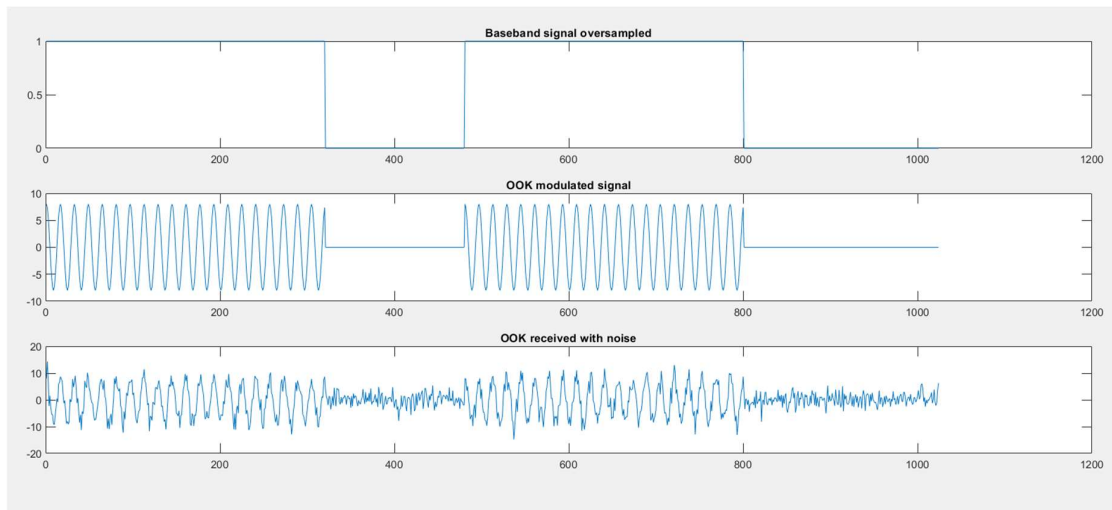


Figure 3.3.2 - OOK modulated and with noise (change this graph)

Firstly, for OOK modulation, as the baseband oversampled signals are generated after the data is being sampled, the amplitude changes when the data is at bit 1, and it will remain the same when the data is at bit 0. Moreover, to simulate the noise on the channel during transmission, we added noise to the OOK modulated signal, which mimics real-life situations which can be seen in Figure 3.3.2.

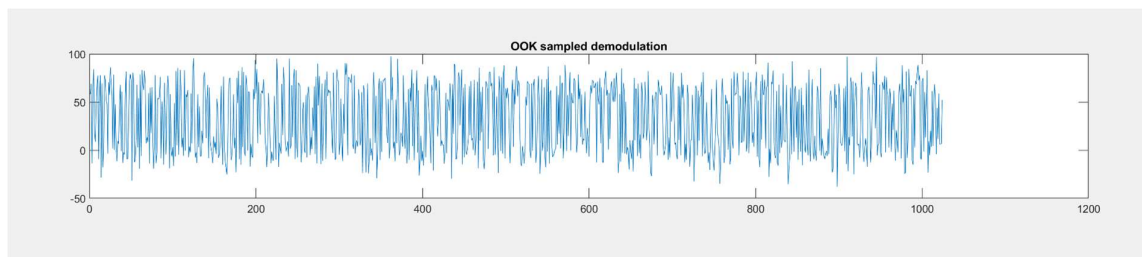


Figure 3.3.3 - OOK demodulation

The OOK sampled demodulation is in Figure 3.3.3. In order to detect and retrieve the original data, the threshold for OOK (average power) is $\frac{A^2}{2}$, which allows the threshold detection to know which data is considered as 1 or 0. If the demodulated signal exceeds the threshold, the signal will consider the output as 1 and if it falls below the threshold, it will consider the output as 0. Hence, it will give us the decoded output of the OOK's coherent detection.

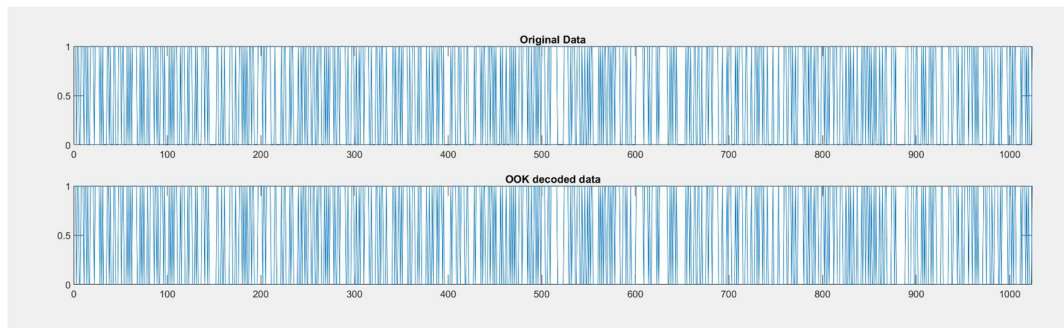


Figure 3.3.4 - OOK original data and decoded data

Now, we are able to compare the decoded data obtained from the OOK coherent detection to the original data. We noticed that the original data and the decoded data are very similar. This shows that our experiment has been proven to stimulate and process the OOK modulation technique correctly in Figure 3.3.4.

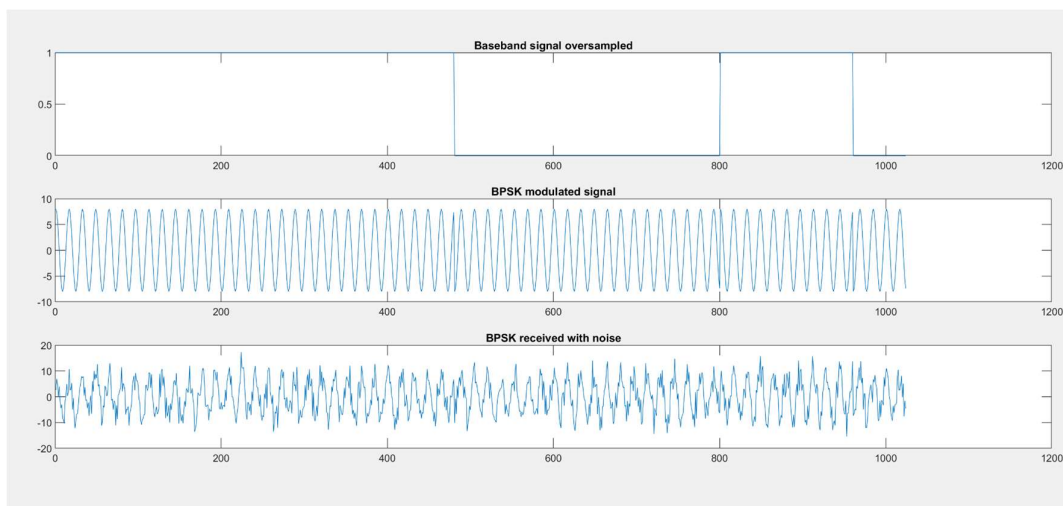


Figure 3.3.5 - BPSK modulated and with noise

Next, for BPSK modulation, as the baseband oversampled signals are generated after the data is being sampled, the phase shifts at 180 degrees, when there is a transition from 0 to 1 bit. Moreover, similar to BPSK, to simulate the noise on the channel during transmission, we added noise to the BPSK modulated signal, which mimics real-life situations as seen in Figure 3.3.5.

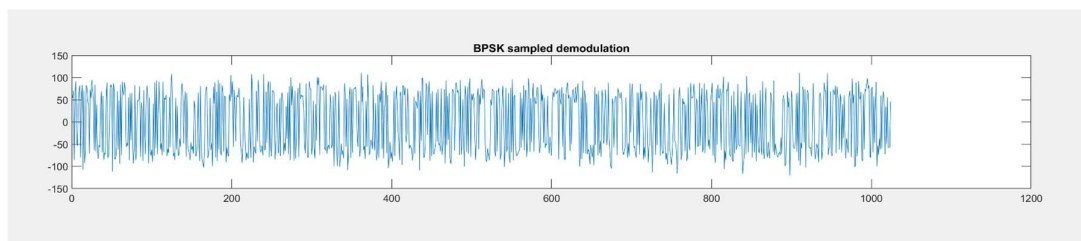


Figure 3.3.6 - BPSK demodulation

The BPSK sampled demodulation is shown in Figure 3.3.6. In order to detect and retrieve the original data, the threshold for BPSK is 0, which allows the threshold detection to know which data is considered as 1 or 0. If the demodulated signal exceeds the threshold, the signal will consider the output as 1 and if it falls below the threshold, it will be considered as 0. Hence, it will give us the decoded output of the BPSK's coherent detection.

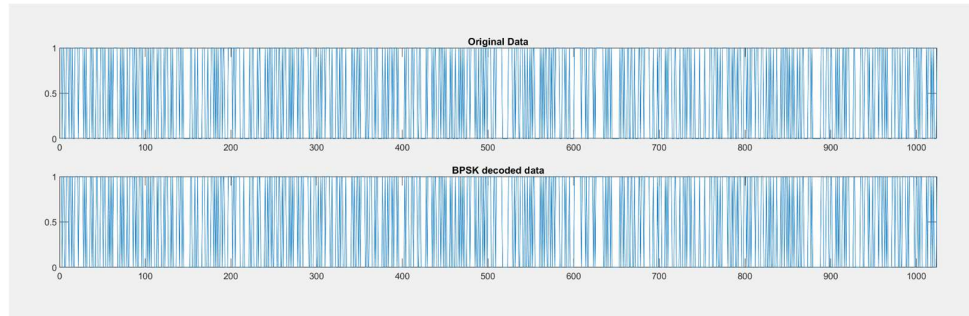


Figure 3.3.7 - BPSK original data and decoded data

Now, we are able to compare the decoded data obtained from the BPSK coherent detection with the original data. We noticed that the original data and the decoded data are very similar. This shows that our experiment has been proven to simulate and process the BPSK modulation technique correctly as seen in Figure 3.3.7.

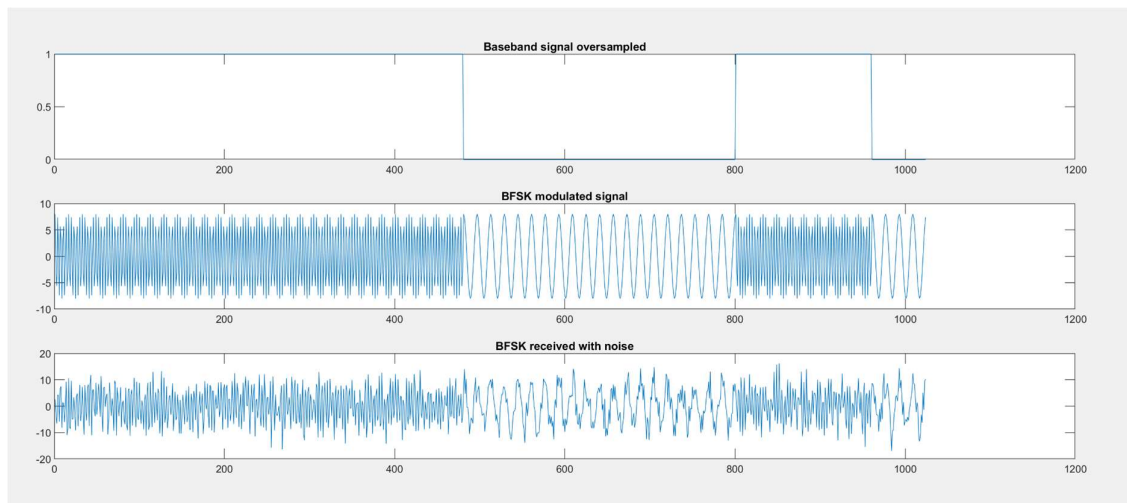


Figure 3.3.8 - BFSK modulated and with noise

Lastly, For BFSK modulation, as the baseband oversampled signals are generated after the data is being sampled, the frequency changes when the data is at bit 1, and it will remain the same when the data is at bit 0, with two carrier signals of 10kHz and 50kHz. Moreover, to simulate the noise on the channel during transmission, we added noise to the BFSK modulated signal, which mimics real-life situations as seen in Figure 3.3.8.

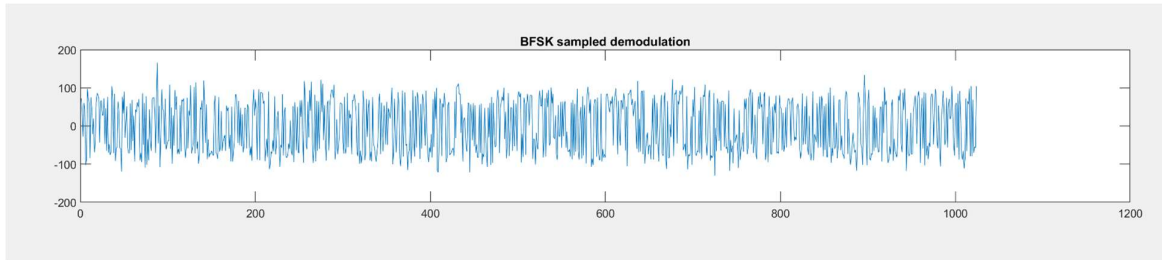


Figure 3.3.9 - BFSK demodulation

The BFSK sampled demodulation is shown in Figure 3.3.9. For BFSK demodulation, it has the same threshold as BPSK, which is 0, and with that, it will give us the decoded output of the BFSK coherent detection.

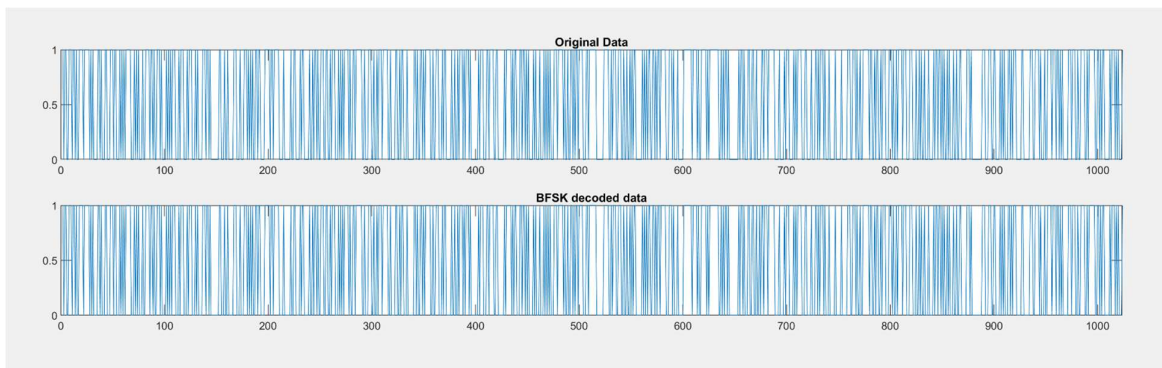


Figure 3.3.10 - BFSK original data and decoded data

Now, we are able to compare the decoded data obtained from the BFSK coherent detection to the original data. We noticed that the original data and the decoded data are very similar. This shows that our experiment has been proven to simulate and process the BFSK modulation technique correctly which is shown in Figure 3.3.10.

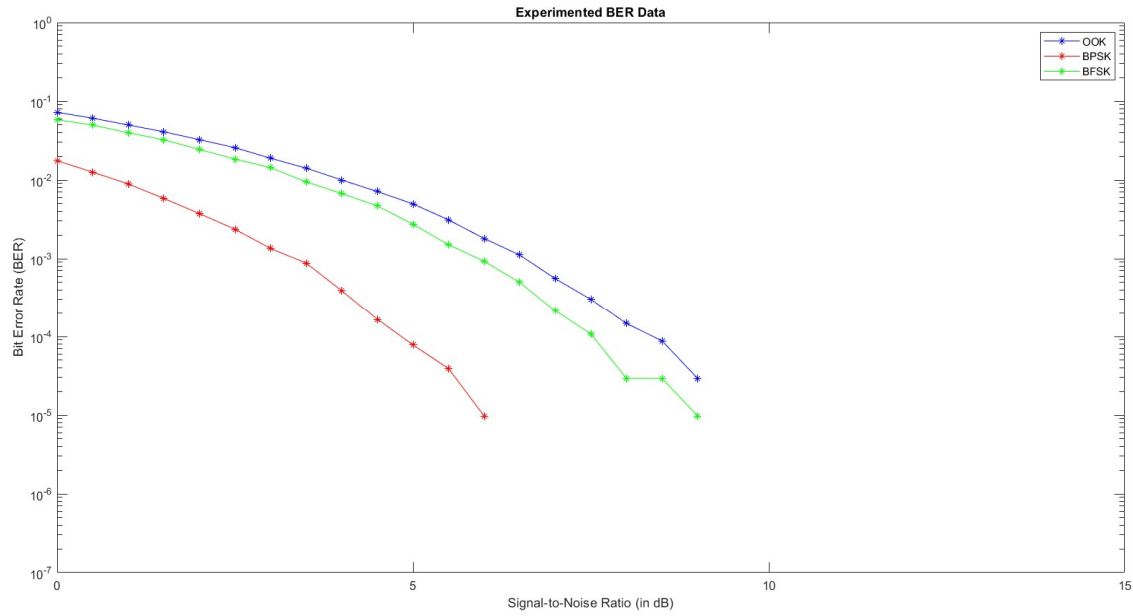


Figure 3.3.11 -Measured bit error rate

Finally, from our experiment, we will generate each of the noise samples from the **range of 0 - 15dB with an interval of 0.5dB**. Next, we obtained the respective bit error rates for all three modulation techniques and we calculated the average bit error rates with the same noise samples from above. We represented our measured bit error rate using MATLAB's semilogy function, which is shown in Figure 3.4.1.

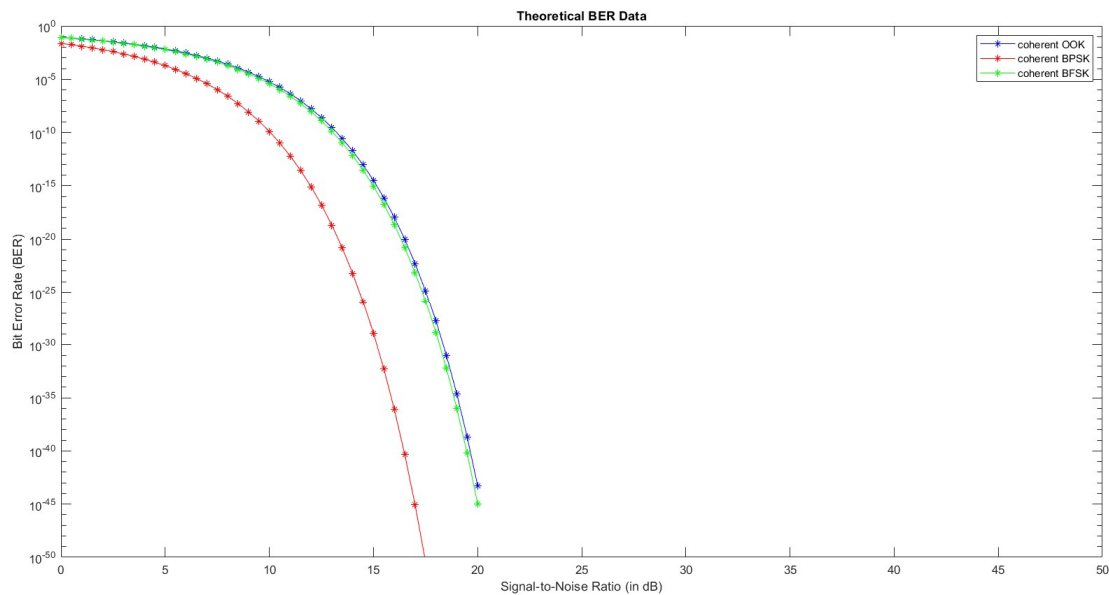


Figure 3.3.12 -Theoretical bit error rate

As seen from the semilogy above, our BPSK signal requires the least signal power for transmission and detection compared to its counterparts. For example, to achieve a bit error rate of 10^{-2} , the required SNR for BPSK is only 1dB, whilst for OOK and BFSK they require 3dB and 4dB respectively. Once we have gotten our measured bit error rates, we plotted a semilogy for the theoretical bit error rates and compared our readings with the theoretical readings.

Comparing the two semilogy plots, it is evident that our measured BER data is similar to the theoretical BER data, which emphasises that our experiment is accurate, which is shown in the experimented and theoretical combined graph below.

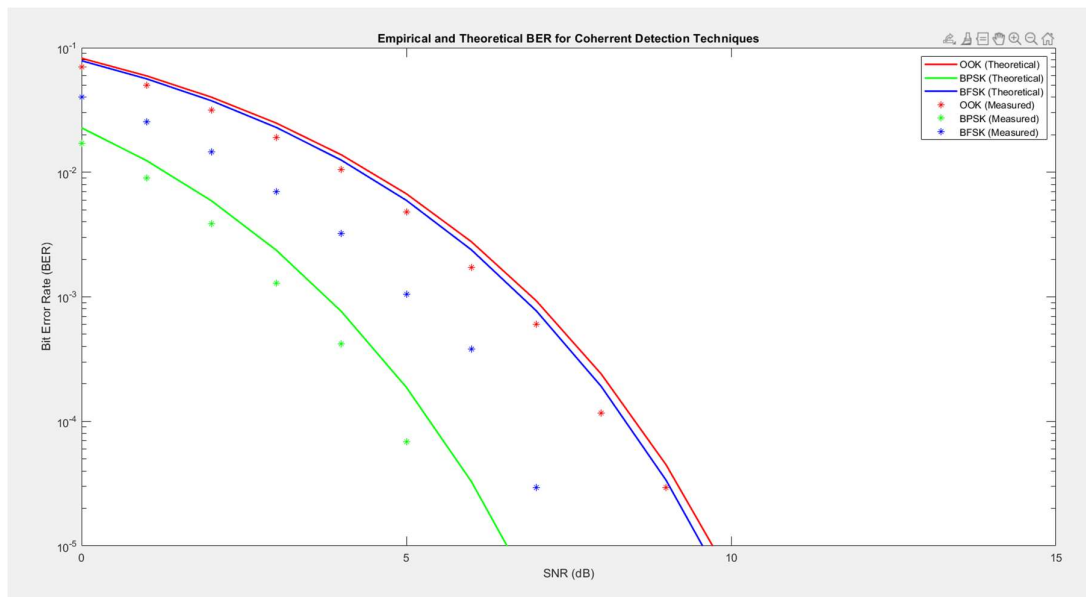


Figure 3.3.13 - Experimented and Theoretical bit error rate (amplitude = 20)

All things considered, we have implemented a functional communication system that does modulation and demodulation for our baseband process. In Phase 3, we will make improvements to the system by testing out different experiments to see which experiment works best.

4. Phase 3 – Basic error control coding

4.1. Introduction

Phase 3 is a continuation of Phase 2. In this phase, we added error control bits in an effort to detect and fix the error codes during transmission. We tested using two different encoding methods, **Cyclic Code** and **Hamming Code**. Generator polynomial ($g(x)$), parity-check matrix ($h(x)$), and the syndrome decoding table (s) are implemented in Cyclic coding. Most of the features mentioned above are implemented through **MATLAB's inbuilt functions**.

The two coding methods are chosen as information about other encoding methods (e.g. Reed-Solomon encoding), despite having **disadvantages like lack of ability to detect burst-errors**, as both are covered in lectures, and easier to check for potential errors manually.

Theoretically, the **Bit-Error Rate (BER) performance** is ranked in the following order, from most to least:

- 1) Unencoded OOK
- 2) Encoded OOK
- 3) Encoded BFSK
- 4) Encoded BPSK

As OOK is just Binary ASK, the theoretical difference between Encoded OOK and Encoded BFSK is expected to be **minimal**, based on Phase 2 results.

Before implementation and experimentation, the definition of the Cyclic Code has to be known. The screenshot on the right is extracted from Slide 74 of Chapter 6: Channel Encoding for Error Control:

🔴 **An (n,k) linear code C is called cyclic if every cyclic shift of the code vector is also a code vector in C .**

☒ That is if $\mathbf{v} = (v_0, v_1, v_2, \dots, v_{n-1})$ is a code word in C , then $\mathbf{v}^1 = (v_{n-1}, v_0, v_1, v_2, \dots, v_{n-2})$ is also a code word, where $\mathbf{v}^1$ is obtained by cyclically shifted one place to the right.

Also, Hamming Codes have a $d_{min} = 3$, and thus $t = 1$. This means that a **single error can always be corrected** regardless of the number of parity-check bits. Given (n, k) Hamming code, and $m = n - k$ parity-check bits, for $n = 2^m - 1$, $m \geq 3$ is required.

Both assumptions could impact the results obtained in the experiments.

4.2. Implementation & Experiment

Implementation

As Phase 3's aim is to add encoding over Phase 2, most codes are left unchanged. The main modifications are listed below:

- 1) All experiments will be generated using noise samples **between 0 and 15dB**, with unit **intervals of 0.5dB**.
- 2) Unencoded OOK, encoded OOK, encoded BFSK and encoded BPSK are implemented for comparison. Each code will be **repeated 100 times**, similar to Phase 2.
- 3) Variables *codeword_length* (n) and *message_length* (k) were added for the **(n,k) configuration** required for error control coding.
- 4) To make encoded codes the same encoded bit count as the unencoded codes (*message bits*), variable *extended_bits* was created.
 - $extended_bits = bits * codeword_length / message_length$
- 5) Some variables and function calls were added to ensure accuracy.
- 6) To distinguish encoded and unencoded data, the **"orig_" prefix** is added for the **unencoded** data manipulation. This is used for BER comparison between unencoded and encoded OOK.

Experiment 1: Using Hamming Codes in (3,1) configuration

The first 3 Hamming Codes are proven to be (3,1), (7,4) and (15,11) in (n,k) configuration², using the formula $n = 2^m - 1$, where $n = k + m$, where *n* is the number of codewords, *k* is the number of message bits, and *m* is the number of parity bits.

For the first experiment, (3,1) is implemented. This is due to $m = 3 - 1 = 2$, which **fails to meet** $m \geq 3$ requirement. Through this **triple repetition code**, it would severely limit the transmission rate to $R = \frac{k}{n} = \frac{1}{3} \approx 0.333$, but might have the **lowest Bit Error Rate (BER)** in any Hamming Code configuration, albeit with **low bandwidth efficiency**.

Experiment 2: Using Hamming Codes in (7,4) configuration

As shown above, (7,4) is the next hamming code configuration. This is the first configuration that **meets** the $m \geq 3$ requirement. The transmission rate has been **improved** to $R = \frac{k}{n} = \frac{4}{7} \approx 0.571$. As bandwidth efficiency increases, the **power efficiency decreases**. Therefore, the BER might increase as a result.

Experiment 3: Using Hamming Codes in (6,4) configuration

This experiment is used to show the **violation of Hamming Code rules**, as $n = 6 = 2^m - 1$, implying that $m = \log_2 7 \approx 2.807$. Since *m* is not an integer, it would mean that there would be errors undetected or uncorrected, likely resulting in **BER higher** than that of Experiment 2.

² <https://www.sciencedirect.com/topics/engineering/hamming-code>

Experiment 4: Using Cyclic Codes in (7,4) configuration

In a Tel Aviv University School of Computer Science Lecture Document³, it has been proven that **Hamming Code is Cyclic**. As such, we use Cyclic Code to prove that the BER is similar to Hamming Code. We use (7,4) configuration for this experiment, as it is a balance between bandwidth efficiency and power efficiency.

However, as Cyclic Code and Hamming Codes run separately, a certain **margin of error is to be expected**, as random codewords are generated on a run-by-run basis.

Experiment 5: Using Cyclic Codes in (6,4) configuration

This experiment attempts to **repeat Experiment 3**, using Cyclic Codes instead. It should result in the **BER higher** than Experiment 4, as explained earlier.

4.3. Results

Result 1: Using Hamming Codes (3, 1) configuration

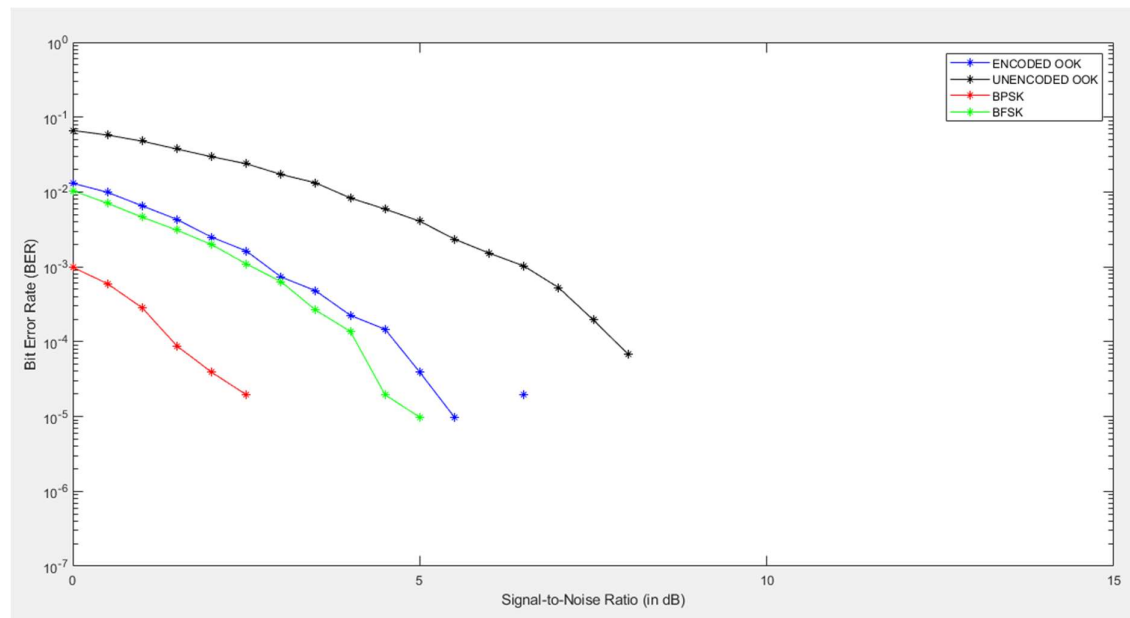


Figure 4.3.1 - Measured Output for Hamming (3,1) configuration

The first Hamming code is (3,1). As shown here, the gain is still substantial, with approximately 4dB gain for encoded OOK and encoded BFSK compared to unencoded OOK, and approximately 6dB gain for encoded BPSK compared to unencoded OOK.

However, to get the gains required, a total of 3072 codeword bits had to be transmitted, as 2048 bits were used for parity bits, leaving just 1024 message bits. This is very inefficient from a transmission bandwidth standpoint, which is improved in the next hamming code (7,4).

³ <http://www.cs.tau.ac.il/~amnon/Classes/2017-ECC/Lectures/Lecture2.pdf>

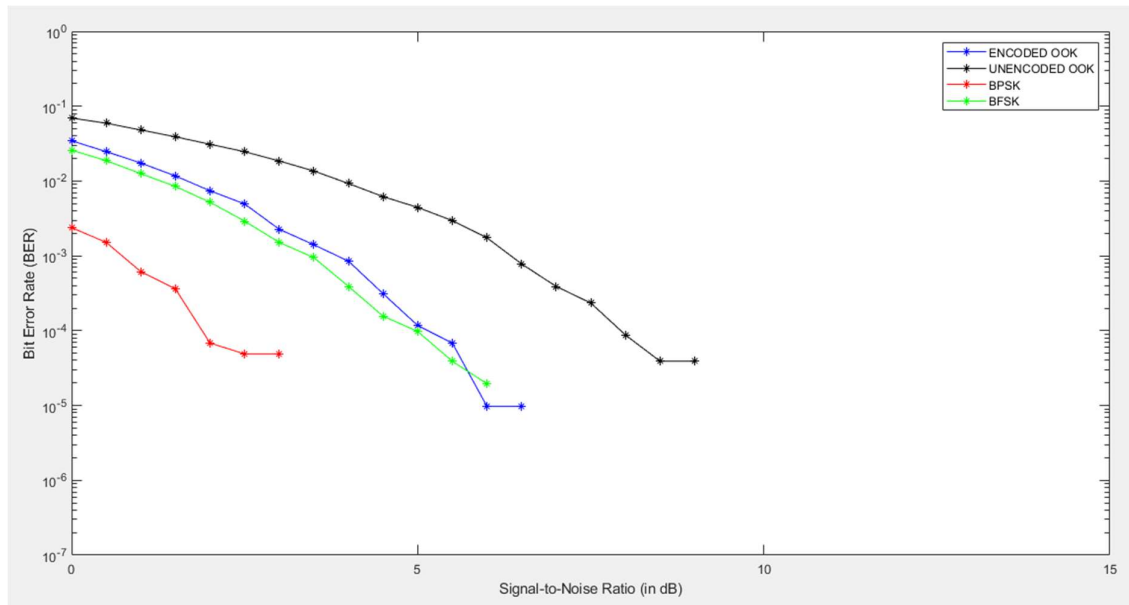
Result 2: Using Hamming Codes in (7,4) configuration

Figure 4.3.2 - Measured Output for Hamming (7,4) configuration

Compared to Hamming Code (3,1), the SNR required increased by approximately 1dB for the same BER performance. The BER performance is offset by the reduction of codewords transmitted. For the same 1024 message bits, we only require a total of 1792 codeword bits, instead of 3072 codeword bits, which is a significant 1280 bits (41.67%) lower as compared to (3,1) configuration.

Further enhancements in bandwidth efficiency can be achieved by (15, 11) and beyond, but if the actual message length required is less than k bits, bit padding is required, wasting transmission bandwidth.

Result 3: Using Hamming Codes in (6,4) configuration

```

5      %Self-defined: Codeword length (n) & Message length (k)
6      codeword_length = 6;
7      message_length = 4;
8
9      %carrier signal 16 times oversampled:
10     samplingFreq = 16* carrierFreq; %sampling frequency is 16 times
11
12     %baseband data rate:
13     dataRate = 1000; %1kbps
14
15     %number of databits:
16     bits = 1024;

```

Command Window

New to MATLAB? See resources for [Getting Started.](#)

Error using `encode` (line 109)
The specified codeword length and message length are not valid.

Figure 4.3.3 - Error Output for Hamming (3,1) configuration

```

if contains(method, 'hamming')
    % hamming code.
    m = n - k;
    if 2^m - 1 ~= n
        error(message('comm:encode:InvalidNKLen'))
    end
end

```

Figure 4.3.4 - Code that invoked the Error Output in Figure 4.3.3

The computation could not be completed due to the error occurring as shown in Figure 4.3.3, as (6,4) configuration computes the following: $m = n_{actual} - k = 6 - 4 = 2$, and $n_{calculated} = 2^2 - 1 = 3$. Since $n_{actual} \neq n_{calculated}$, this invoked the error found in `encode.m`.

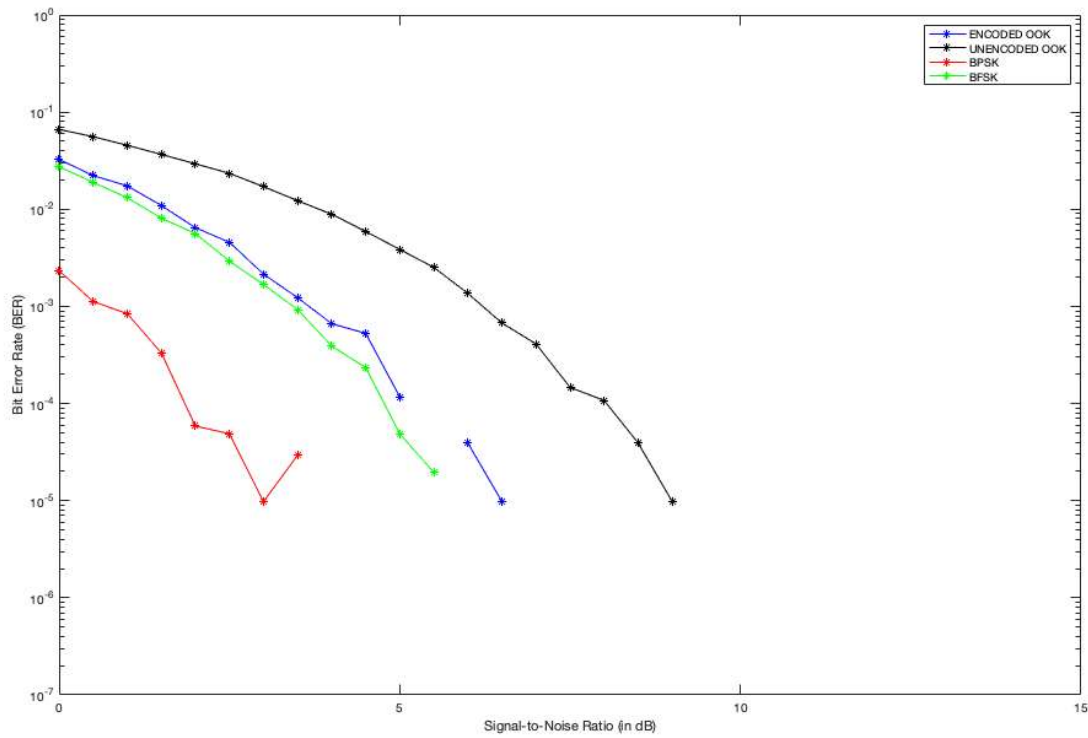
Result 4: Using Cyclic Codes in (7,4) configuration

Figure 4.3.5 - Measured Output for Cyclic (7,4) configuration

The output is similar to the measured output for Experiment 2, as Hamming Code is also a cyclic code. However, the BER rate might still be slightly higher than Hamming Code, which means that Hamming Code is more efficient.

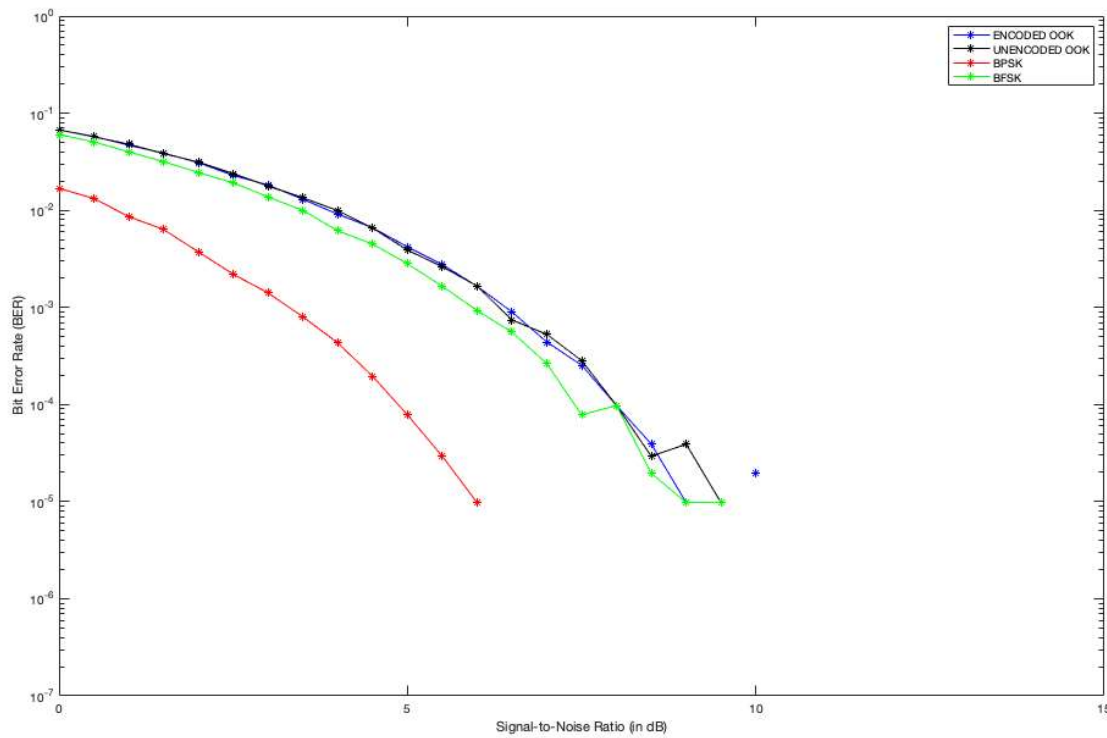
Result 5: Using Cyclic Codes in (6,4) configuration

Figure 4.3.6 - Measured Output for Cyclic (6,4) configuration

The measured output for Cyclic Codes in (6,4) configuration yielded much worse BER for encoded OOK, encoded BFSK and encoded BPSK compared to Cyclic Code in (7,4) configuration. This is in line with expectation, as in (6,4) configuration, only 2 bits are used as parity bits, which means only 1 bit is used for detection of an error, and no bit is available for error correction.

Therefore, there is almost 0dB gain comparing unencoded OOK and encoded OOK. Since encoded OOK and encoded BFSK has similar gains based on theory and previous experiments, it is unsurprising that the encoded BFSK have approximately 0dB gain compared to unencoded OOK as well.

5. Conclusion

From this study, we have successfully designed a basic digital communication system in MATLAB, incorporating multiple aspects and techniques.

In Phase 1, a simple digital communication system was designed to simulate and analyse the effect of additive white Gaussian noise channels with varying Signal-to-Noise (SNR) values on a Baseband data. A Baseband data signal is transmitted through the noise channel and the distorted signal is then reconstructed and compared with the original Baseband data to compute the Bit Error Rate (BER). The relationship between BER and varying SNR values was then observed whereby as the value of SNR increases, the BER decreases.

In Phase 2, three different modulation schemes are experimented with to understand the process of baseband to band-pass conversion for modulation. Modulation is a process of encoding information from a message source in a manner that is suitable for transmission. It usually involves the translation of a baseband to a modulated signal at higher frequencies. Modulation is required to ensure that it is robust against noise and interference, ease of radiation, and simultaneously transmit several signals. The process usually involves modulating the amplitude, frequency, and/or phase of a carrier sine wave. For our experiment, we compared the three basic signal modulation techniques, Binary ASK (also known as On-Off Keying method), Binary FSK, and Binary PSK. Overall, it is apparent that BPSK modulation has the lowest calculated BER, followed by BFSK, and OOK, at a given SNR value,

In Phase 3, using the previous knowledge, we applied channel coding to provide error detection and error correction. In this phase, we added Hamming Codes and Cyclic Codes encoding techniques and experimented with different codeword and message lengths to find its effect on bandwidth and power efficiency. Similar to Phase 2, BPSK has the best BER performance. However, if there are uncorrectable errors, higher transmission power is required to achieve the same BER performance.

It is noted that this study excludes Source Coding, which could potentially improve the bandwidth efficiency. The lossless compression techniques like Huffman Codes could have resulted in reduced BER, as more raw data is transmitted with the same bandwidth.

In conclusion, from the three phases, we were able to show that the BER can be reduced by employing various modulation and channel coding techniques that we have learned during this course.

6. References

- <https://www.rfwireless-world.com/Terminology/OOK-vs-FSK-vs-ASK.html>
- Lecture Notes